

해밍 부호의 오류 정정 아이디어

이 문서는 선형 블록부호로서의 **해밍(Hamming) 부호**가 **단일 비트 오류**를 어떻게 **위치까지 특정하여 정정**하는지, 그 직관과 Hamming(7,4)에서의 구체적 규칙을 정리한다. 실습 코드 `hamming_code_practice.py`와 동일한 비트 배치(위치 1,2,4는 검사비트, 짝수 패리티)를 기준으로 서술한다.

1. 왜 단순 패리티만으로는 부족한가

한 개의 **전체 패리티 비트**를 덧붙이면, 수신단에서 “전체 1의 개수의 짝/홀” 검사로 **오류 발생 여부**는 알 수 있다. 그러나 **어느 위치가 틀렸는지**는 알 수 없어, 비트를 되돌리는 **정정**까지는 할 수 없다.

해밍 부호의 핵심은, 검사 비트를 **여러 개** 두되 각 검사가 **서로 다른 위치들의 부분집합**을 담당하게 하여, 검사 결과의 조합이 **오류 위치의 번호**를 가리키게 만드는 것이다.

2. 해밍 부호의 한 줄 요약

여러 개의 직교에 가까운 **패리티 검사**를 겹쳐 두고, 수신어에서 각 검사의 만족/불만족을 이진수로 묶은 값을 **신드롬(syndrome)**이라 부른다. 채널에서 **최대 1비트만 반전**했다고 가정하면, 이 신드롬은 **반전된 비트의 인덱스(위치)**와 같게 설계할 수 있다. 그 위치의 비트만 뒤집으면($0 \leftrightarrow 1$) 원래의 합법적인 코드워드로 돌아간다.

3. 비트 위치 배치: Hamming(7,4)

코드 길이 $n = 7$, 정보 비트 수 $k = 4$ 인 **(7,4) 해밍 부호**는 흔히 다음과 같이 **위치 번호 1부터 7**을 쓴다.

위치	1	2	3	4	5	6	7
역할	검사 p_1	검사 p_2	정보 d_1	검사 p_4	정보 d_2	정보 d_3	정보 d_4

규칙: 위치 번호가 2의 거듭제곱(1, 2, 4)인 칸에는 **검사비트**, 나머지 칸에는 **정보비트**를 둔다.

4. 검사 집합과 짝수 패리티

각 검사비트는 “자기 자신을 포함한 특정 위치들”의 XOR이 0이 되도록(짝수 패리티) 정해진다. 표준적인 배치는 **위치 번호의 이진 표현**으로 검사 집합을 정한다.

- **검사 1** (p_1 , 위치 1): 위치 번호의 **맨 아래 비트가 1** 인 곳 — $\{1, 3, 5, 7\}$
- **검사 2** (p_2 , 위치 2): **둘째 비트가 1** 인 곳 — $\{2, 3, 6, 7\}$
- **검사 4** (p_4 , 위치 4): **셋째 비트가 1** 인 곳 — $\{4, 5, 6, 7\}$

송신 시에는 정보비트 d_1, d_2, d_3, d_4 가 정해지면, 각 검사 집합의 XOR이 0이 되도록 p_1, p_2, p_4 를 유일하게 결정하면 된다. (이때 XOR은 GF(2)에서의 덧셈과 같다.)

정보가 위치 3,5,6,7에 각각 d_1, d_2, d_3, d_4 일 때, 짝수 패리티 조건은 다음과 같다.

$$p_1 \oplus d_1 \oplus d_2 \oplus d_4 = 0$$

$$p_2 \oplus d_1 \oplus d_3 \oplus d_4 = 0$$

$$p_4 \oplus d_2 \oplus d_3 \oplus d_4 = 0$$

따라서

$$p_1 = d_1 \oplus d_2 \oplus d_4, \quad p_2 = d_1 \oplus d_3 \oplus d_4, \quad p_4 = d_2 \oplus d_3 \oplus d_4.$$

5. 신드롬과 오류 위치

수신한 7비트를 $r_1, \dots, r_7$ 이라 하자. 각 검사를 다시 계산할 때, **짝수 패리티를 만족하면 0, 만족하지 않으면 1** 이라고 하면 다음과 같이 정의할 수 있다.

$$s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7$$

$$s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7$$

$$s_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7$$

신드롬 값 을 이진수 $(s_4 s_2 s_1)_2$ 로 읽어 십진 정수 s 로 두면,

- $s = 0$: 세 검사 모두 만족 $\rightarrow$ **오류 없음** 으로 판단(단일 비트 오류 모델에서).
- $s \in \{1, \dots, 7\}$: **위치 s 의 비트가 반전** 되었다고 보고, 그 비트만 XOR 1로 뒤집어 **정정** 한다.

직관적으로, 위치 j 의 비트가 틀리면 j 의 이진 표현에서 1인 자리에 해당하는 검사들만 깨지므로, (s_4, s_2, s_1) 가 바로 j 의 이진 표현이 된다.

6. 정정 이후 정보 복원

(7,4) 배치에서는 정보비트가 항상 위치 3, 5, 6, 7에 있으므로, 정정이 끝난 코드워드에서 이 네 위치를 순서대로 읽으면 원래 송신하려던 4비트 정보를 얻는다.

7. 한계: 다중 비트 오류

해밍(7,4)의 신드롬 디코딩은 보통 **최대 1비트 오류** 를 정정하도록 설계된다.

- **2비트 이상**이 동시에 바뀌면 신드롬이 0이 아닐 수 있지만, 그 값이 **실제 오류 패턴과 맞지 않아** 엉뚱한 위치를 고치는 **오정정(miscorrection)** 이 발생할 수 있다.
- 어떤 2비트 오류는 신드롬이 0이 되어 **검출도 되지 않는** 경우가 있다.

실무에서는 전체 패리티 비트를 하나 더해 **단일 정정·이중 검출(SECDED)** 등으로 확장하기도 한다.

8. 정리

항목	내용
목표	단일 비트 반전 시 위치 특정 + 정정
수단	겹치는 패리티 검사 여러 개
수신 처리	검사 불일치 패턴 → 신드롬 → 해당 위치 비트 반전
전제	채널에서 한 번에 틀리는 비트가 많아지지 않음(또는 확률이 매우 낮음)

같은 폴더의 `hamming_code_practice.py` 에서 위 규칙으로 부호화·신드롬 계산·정정을 직접 입력해 볼 수 있다.